

1. La función principal de la capa de transporte es proporcionar comunicación lógica entre procesos de aplicación que se ejecutan en diferentes hosts. Se basa en el servicio de la capa de red (que comunica hosts) y lo amplía mediante técnicas de multiplexación y demultiplexación para asegurar que los mensajes lleguen a la aplicación o socket correcto.
2.
 - a. Segmento TCP: Es complejo y estructurado. Contiene puerto de origen, puerto de destino, número de secuencia, número de reconocimiento, longitud de cabecera, bits reservados, indicadores (URG, ACK, PSH, RST, SYN, FIN), tamaño de ventana (para el control de flujo), checksum (suma de comprobación), puntero de datos urgentes, un campo opcional y los datos.
 - b. Segmento UDP: Es minimalista, constando de solo 4 campos en su cabecera (8 bytes en total): puerto de origen, puerto de destino, longitud (tamaño total) y checksum (suma de comprobación), seguidos por los datos de la aplicación.
3. El objetivo de los puertos es identificar de forma unívoca a un proceso receptor (socket) específico dentro de un host determinado. Junto con la dirección IP, permiten a la capa de transporte realizar la multiplexación y demultiplexación, asegurando que los datos que llegan desde la red sean entregados a la aplicación correspondiente.
4.
 - a. Confiabilidad: TCP es totalmente confiable, garantizando que los datos lleguen completos, en orden y sin errores. UDP no es confiable ("best-effort"); los paquetes pueden perderse, corromperse o llegar desordenados.
 - b. Multiplexación: Ambos protocolos proveen los servicios de multiplexación y demultiplexación hacia las aplicaciones.
 - c. Orientado a la conexión: TCP es orientado a la conexión, requiriendo un acuerdo previo entre procesos antes de transmitir datos. UDP es sin conexión, iniciando la transferencia de forma directa sin acuerdos preliminares.
 - d. Controles de congestión: TCP incluye control de congestión, frenando la tasa de transmisión del emisor si detecta que la red está sobrecargada. UDP no tiene control de congestión, y envía datos a cualquier velocidad sin importar la saturación de la red.
 - e. Utilización de puertos: Ambos utilizan puertos para comunicarse con las aplicaciones. Sin embargo, en TCP la conexión se demultiplexa exigiendo una coincidencia exacta de cuatro elementos (IP origen, puerto origen, IP destino, puerto destino), mientras que en UDP la

demultiplexación depende únicamente del puerto destino y la IP destino.

5. La unidad de datos (PDU) de la capa de transporte se denomina formalmente "segmento". Sin embargo, el término "datagrama" se utiliza con frecuencia para referirse específicamente a los paquetes del protocolo UDP (User Datagram Protocol). Por convención y para evitar confusiones con los "datagramas" de la capa de red (IP), los libros técnicos prefieren llamar a ambos "segmentos".
6. El saludo de tres vías es el proceso para establecer una conexión TCP y consta de estos pasos:
 - a. El cliente envía un segmento SYN (sin datos) indicando su intención de conexión y su número de secuencia inicial.
 - b. El servidor recibe el SYN y responde con un segmento SYNACK, concediendo la conexión, informando su propio número de secuencia inicial y reconociendo el del cliente.
 - c. El cliente recibe el SYNACK y envía un segmento ACK al servidor para confirmar que la conexión ha sido establecida (este segmento ya puede transportar datos). UDP no posee esta característica, dado que es un protocolo sin conexión y transmite de inmediato.
7. El ISN es el primer número de secuencia (generado de forma aleatoria) que un host le asigna al primer byte de su flujo de datos. Su relación con el saludo de tres vías es directa: es durante esta fase de negociación inicial cuando el cliente y el servidor se comunican sus respectivos ISN a través de los paquetes SYN y SYNACK, inicializando las variables de estado antes de intercambiar cualquier dato útil.
8. El MSS (Maximum Segment Size) es la máxima cantidad de datos de la capa de aplicación que el emisor puede encapsular en un único segmento TCP sin que deba ser fragmentado (su valor se determina según el MTU del enlace). Este valor se negocia durante el establecimiento de la conexión TCP (el saludo de tres vías), y la información se incluye específicamente utilizando el campo de longitud variable de "Opciones" en la cabecera del segmento TCP.
9.
 - a. 172.28.0.1:56124
 - b. 172.20.243.226%eth0:bootpc

```
root@debian:/home/redes# ss -tl
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	128	0.0.0.0:ssh	0.0.0.0:*
LISTEN	0	128	127.0.0.1:ipp	0.0.0.0:*
LISTEN	0	5	127.0.0.1:4038	0.0.0.0:*
LISTEN	0	128	:::ssh	:::*
LISTEN	0	128	:::1:ipp	:::*
LISTEN	0	4096	:::1:50051	:::*
c. LISTEN	0	4096	:::ffff:127.0.0.1:50051	:::*

```

root@debian:/home/redes# ss -ul
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port
UNCONN     0            0            127.0.0.1:4038          0.0.0.0:*
UNCONN     0            0            0.0.0.0:34976          0.0.0.0:*
UNCONN     0            0            0.0.0.0:631            0.0.0.0:*
UNCONN     0            0            0.0.0.0:mdns            0.0.0.0:*
UNCONN     0            0            [::]:mdns               [::]:*
UNCONN     0            0            [::]:38230              [::]:*

root@debian:/home/redes# ss -tp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
ESTAB      0            0            172.28.0.1:56124        172.28.0.90:imap2      users:(( "th
underbird",pid=80326,fd=60))
root@debian:/home/redes# ss -up
Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
0            0            172.20.243.226:eth0:bootpc 172.20.240.1:bootps    users:(( "Ne
tworkManager",pid=568,fd=23))
root@debian:/home/redes# ss -tlp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            128          0.0.0.0:ssh             0.0.0.0:*               users:(( "sshd",pid=674,fd=3))
LISTEN     0            128          127.0.0.1:ipp            0.0.0.0:*               users:(( "cupsd",pid=93306,fd=7))
LISTEN     0            5           127.0.0.1:4038          0.0.0.0:*               users:(( "core-daemon",pid=651,fd=5))
LISTEN     0            128          [::]:ssh                 [::]:*                  users:(( "sshd",pid=674,fd=4))
LISTEN     0            128          [::]:ipp                 [::]:*                  users:(( "cupsd",pid=93306,fd=6))
LISTEN     0            4096         [::]:50051               [::]:*                  users:(( "core-daemon",pid=651,fd=9))
LISTEN     0            4096         [::]:50051               [::]:*                  users:(( "core-daemon",pid=651,fd=10))

root@debian:/home/redes# ss -ulp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
UNCONN     0            0            127.0.0.1:4038          0.0.0.0:*               users:(( "core-daemon",pid=651,fd=6))
UNCONN     0            0            0.0.0.0:34976          0.0.0.0:*               users:(( "avahi-daemon",pid=560,fd=14))
UNCONN     0            0            0.0.0.0:631            0.0.0.0:*               users:(( "cups-browsed",pid=93308,fd=7))
UNCONN     0            0            0.0.0.0:mdns            0.0.0.0:*               users:(( "avahi-daemon",pid=560,fd=12))
UNCONN     0            0            [::]:mdns               [::]:*                  users:(( "avahi-daemon",pid=560,fd=13))
UNCONN     0            0            [::]:38230              [::]:*                  users:(( "avahi-daemon",pid=560,fd=15))

root@debian:/home/redes# netstat -t
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State
tcp    0      0 172.28.0.1:56124    172.28.0.90:imap2  ESTABLISHED

root@debian:/home/redes# netstat -u
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State
udp    0      0 172.20.243.226:bootpc 172.20.240.1:bootps ESTABLISHED

root@debian:/home/redes# netstat -tl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State
tcp    0      0 0.0.0.0:ssh        0.0.0.0:*          LISTEN
tcp    0      0 localhost:ipp       0.0.0.0:*          LISTEN
tcp    0      0 localhost:4038      0.0.0.0:*          LISTEN
tcp6   0      0 [::]:ssh            [::]:*             LISTEN
tcp6   0      0 localhost:ipp       [::]:*             LISTEN
tcp6   0      0 localhost:50051     [::]:*             LISTEN
tcp6   0      0 127.0.0.1:50051    [::]:*             LISTEN

root@debian:/home/redes# netstat -ul
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State
udp    0      0 localhost:4038      0.0.0.0:*          LISTEN
udp    0      0 0.0.0.0:34976      0.0.0.0:*          LISTEN
udp    0      0 0.0.0.0:631        0.0.0.0:*          LISTEN
udp    0      0 0.0.0.0:mdns       0.0.0.0:*          LISTEN
udp6   0      0 [::]:mdns          [::]:*             LISTEN
udp6   0      0 [::]:38230         [::]:*             LISTEN

root@debian:/home/redes# netstat -tp
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State      PID/Program name
tcp    0      0 172.28.0.1:56124    172.28.0.90:imap2  ESTABLISHED 80326/thunderbird

root@debian:/home/redes# netstat -up
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State      PID/Program name
udp    0      0 172.20.243.226:bootpc 172.20.240.1:bootps ESTABLISHED 568/NetworkManager

root@debian:/home/redes# netstat -tlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State      PID/Program name
tcp    0      0 0.0.0.0:ssh        0.0.0.0:*          LISTEN     674/sshd: /usr/sbin
tcp    0      0 localhost:ipp       0.0.0.0:*          LISTEN     93306/cupsd
tcp    0      0 localhost:4038      0.0.0.0:*          LISTEN     651/python
tcp6   0      0 [::]:ssh            [::]:*             LISTEN     674/sshd: /usr/sbin
tcp6   0      0 localhost:ipp       [::]:*             LISTEN     93306/cupsd
tcp6   0      0 localhost:50051     [::]:*             LISTEN     651/python
tcp6   0      0 127.0.0.1:50051    [::]:*             LISTEN     651/python

root@debian:/home/redes# netstat -ulp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address      Foreign Address     State      PID/Program name
udp    0      0 localhost:4038      0.0.0.0:*          LISTEN     651/python
udp    0      0 0.0.0.0:34976      0.0.0.0:*          LISTEN     560/avahi-daemon: r
udp    0      0 0.0.0.0:631        0.0.0.0:*          LISTEN     93308/cups-browsed
udp    0      0 0.0.0.0:mdns       0.0.0.0:*          LISTEN     560/avahi-daemon: r
udp6   0      0 [::]:mdns          [::]:*             LISTEN     560/avahi-daemon: r
udp6   0      0 [::]:38230         [::]:*             LISTEN     560/avahi-daemon: r

```

- 10.** Si llega un segmento TCP con el flag SYN activo a un host y el puerto destino no tiene ningún proceso en estado LISTEN, el host receptor enviará como respuesta un segmento especial de reinicio con el flag RST (Reset) activado. Con este segmento, el receptor le está diciendo al emisor: "No tengo un socket para ese segmento", rechazando así la conexión.

```
root@debian:/home/redes# hping3 -S -p 22 172.20.243.226
HPING 172.20.243.226 (eth0 172.20.243.226): S set, 40 headers + 0 data bytes
^C
--- 172.20.243.226 hping statistic ---
76 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
a.
root@debian:/home/redes# hping3 -S -p 40 172.20.243.226
HPING 172.20.243.226 (eth0 172.20.243.226): S set, 40 headers + 0 data bytes
^C
--- 172.20.243.226 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
b.
```

- c.** El puerto 22 responde concediendo el inicio de la conexión (SYN/ACK), mientras que el puerto 40 responde rechazando la conexión (RST). Si se ejecuta el comando `ss -tl` o `ss -tln` se verá que el puerto 22 se encuentra en estado LISTEN (habitualmente asociado al servicio SSH). Como hay un proceso escuchando y preparado para recibir conexiones, el servidor responde correctamente a su segmento SYN inicial enviando un segmento SYN/ACK para continuar con el saludo de tres vías. Por el contrario, el puerto 40 no aparecerá en la lista de puertos en escucha. Al no existir ningún proceso en estado LISTEN en ese puerto, el protocolo TCP del sistema operativo descarta la petición enviando inmediatamente un segmento RST.

- 11.** Cuando un host recibe un paquete UDP cuyo número de puerto de destino no corresponde a ningún socket UDP activo (es decir, ningún proceso lo está escuchando), el sistema operativo del host descarta el datagrama y responde al emisor enviando un mensaje especial de error ICMP de "puerto de destino inalcanzable" (específicamente, un mensaje ICMP de Tipo 3, Código 3).

```
root@debian:/home/redes# hping3 --udp -p 5353 172.20.243.226
HPING 172.20.243.226 (eth0 172.20.243.226): udp mode set, 28 headers + 0 data bytes
^C
--- 172.20.243.226 hping statistic ---
3 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
a.
root@debian:/home/redes# hping3 --udp -p 40 172.20.243.226
HPING 172.20.243.226 (eth0 172.20.243.226): udp mode set, 28 headers + 0 data bytes
^C
--- 172.20.243.226 hping statistic ---
2 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
b.
```

- c.** Si se ejecuta el comando `ss -ul` se notará que el puerto 5353 sí aparece en la lista (suele estar abierto por el servicio `avahi-daemon` o `mDNS`). Como el puerto está activo, el sistema operativo le entrega el datagrama a la aplicación. Sin embargo, al ser un datagrama vacío o basura generado por `hping3`, la aplicación simplemente lo ignora de forma silenciosa y no

responde nada. Por el contrario, el puerto 40 no aparece en la lista de ss-ul. Como no hay ningún socket esperándolo, el sistema operativo aplica la regla del protocolo mencionada al principio y genera automáticamente el mensaje ICMP de "puerto inalcanzable" para avisarle al emisor del fallo.

12. Los estados principales son:

- a.** CLOSED: Es el estado inicial, donde no existe ninguna conexión activa.
- b.** LISTEN: El servidor ha creado un socket y está a la escucha (esperando) pasivamente solicitudes de conexión de clientes entrantes.
- c.** SYN_SENT: El cliente ha enviado el segmento SYN inicial para solicitar la conexión y está esperando el segmento SYN-ACK de respuesta.
- d.** SYN_RCVD (o SYN_RECEIVED): El servidor ha recibido un segmento SYN, ha respondido con un SYN-ACK, y ahora espera el ACK final del cliente para confirmar el establecimiento de la conexión.
- e.** ESTABLISHED: El saludo de tres vías se ha completado. La conexión está totalmente establecida y ambos extremos pueden enviarse datos de aplicación (payload) de forma bidireccional.
- f.** FIN_WAIT_1: El proceso de la aplicación inicia el cierre de la conexión (cierre activo). Envía un segmento FIN al otro extremo y espera su paquete de reconocimiento ACK.
- g.** FIN_WAIT_2: El extremo que inició el cierre ha recibido el ACK de su propio segmento FIN, pero ahora se queda esperando a que el otro host le envíe su propio segmento FIN para cerrar su lado de la conexión.
- h.** TIME_WAIT: El extremo que inició el cierre ha recibido el FIN del otro host y envía el ACK final. Entra en un tiempo de espera (habitualmente de 30 segundos a 2 minutos, definido como 2MSL) por si su último ACK se pierde y necesita ser retransmitido, tras lo cual pasa a CLOSED liberando los recursos.
- i.** CLOSE_WAIT: El extremo que recibe la solicitud de cierre pasiva (recibe un FIN) envía el ACK y entra en este estado. Aquí, espera a que su propia aplicación local finalice las tareas y ordene cerrar el socket explícitamente.
- j.** LAST_ACK: Tras estar en CLOSE_WAIT y que la aplicación local cierre el socket, el host envía su segmento FIN y espera el último ACK del otro extremo para finalmente ir al estado CLOSED.
- k.** CLOSING: Ocurre poco frecuentemente, en un "cierre simultáneo", cuando ambos extremos envían un segmento FIN casi al mismo tiempo. Un extremo ha enviado su FIN, recibe el FIN del otro, y envía el ACK respectivo, quedando a la espera del ACK de su propio FIN.

13.

- a. Hay 9 conexiones establecidas (ESTAB).
- b. Hay 5 puertos abiertos (LISTEN).
- c. No, mirando las direcciones IP previas a los puertos en Local Address y Peer Address nos damos cuenta de que residen en máquinas diferentes.
- d. Sí, residen en la misma.
- e.
 - **sshd**: Servidor (escucha en el puerto 22 y atiende conexiones entrantes).
 - **apache2**: Servidor (escucha en el puerto 80 para tráfico HTTP).
 - **named**: Servidor (escucha en el puerto 53 para resolución DNS).
 - **postfix**: Servidor (escucha en el puerto 25 para correo SMTP).
 - **x-www-browser**: Cliente (inicia conexiones desde puertos altos y aleatorios hacia puertos 443 y 9500 remotos).
 - **ssh**: Cliente (inicia la conexión desde el puerto 41220 hacia el puerto 22 de la misma máquina).
- f.
 - **Cierre iniciado por el host remoto**: La conexión en estado **CLOSE-WAIT** (IP remota 200.115.89.30:443). Este estado indica que el host remoto envió la petición de cierre (FIN) y el sistema local está esperando que la aplicación local cierre su extremo.
 - **Cierre iniciado por el host local**: La conexión en estado **TIME-WAIT** (IP remota 54.149.207.17:443). Este estado ocurre cuando el host local es quien inició el cierre activo (envió el primer FIN) y ahora está en un período de espera para asegurar que el remoto recibió la confirmación del cierre.
- g. Hay 1 pendiente. Es la que se encuentra en estado SYN-SENT (hacia la IP 43.232.2.2:9500). Este estado indica que el cliente local envió un paquete SYN para iniciar el Three-way Handshake y está esperando el SYN-ACK de respuesta del servidor.

14.

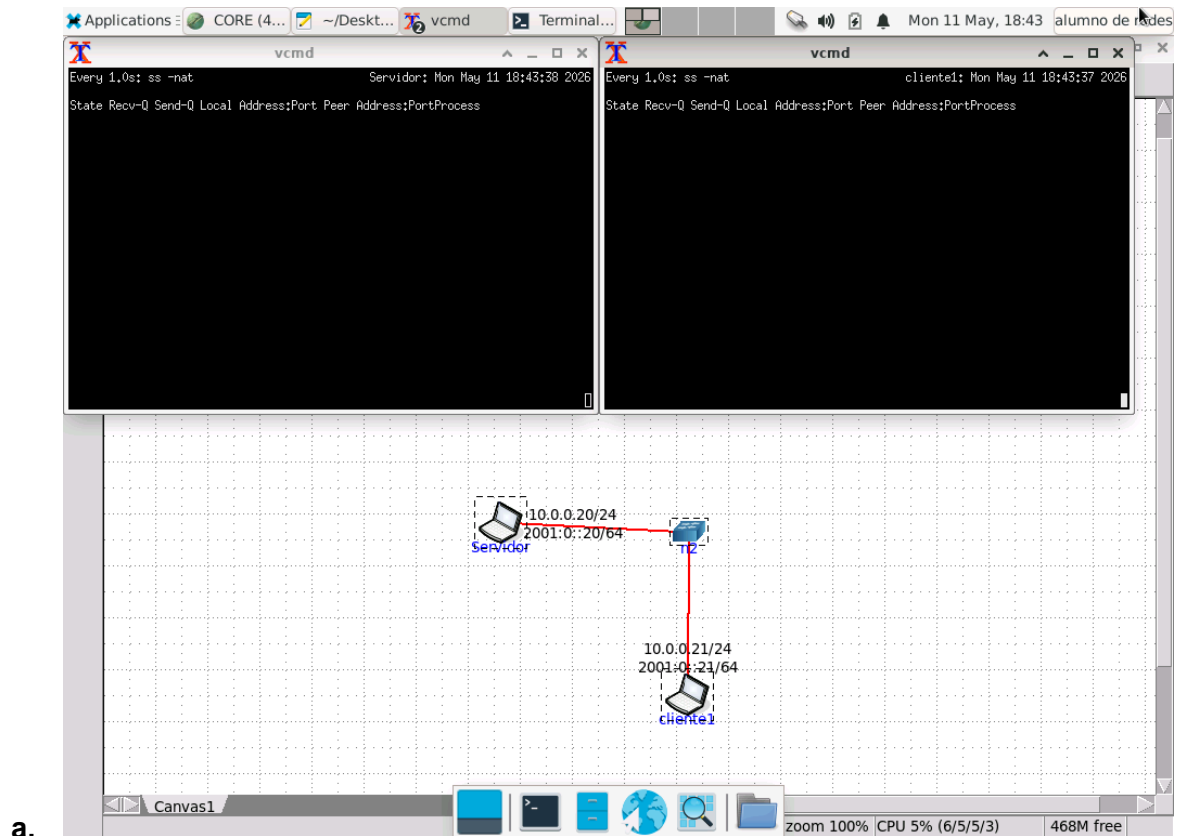
- a.
 - **Llegó al destino**: El segmento **SYN** inicial enviado por el cliente. Podemos afirmarlo porque el servidor se encuentra en estado SYN-RECV, lo que indica que recibió exitosamente la petición de sincronización del cliente y procesó ese primer paso.
 - **Se está perdiendo en la red**: El segmento de respuesta **SYN-ACK** enviado desde el servidor hacia el cliente. Sabemos esto porque el cliente permanece "atascado" en el estado SYN-SENT (con un paquete en su cola de envío, Send-Q 1), esperando una respuesta que no termina de llegar para poder establecer la conexión.

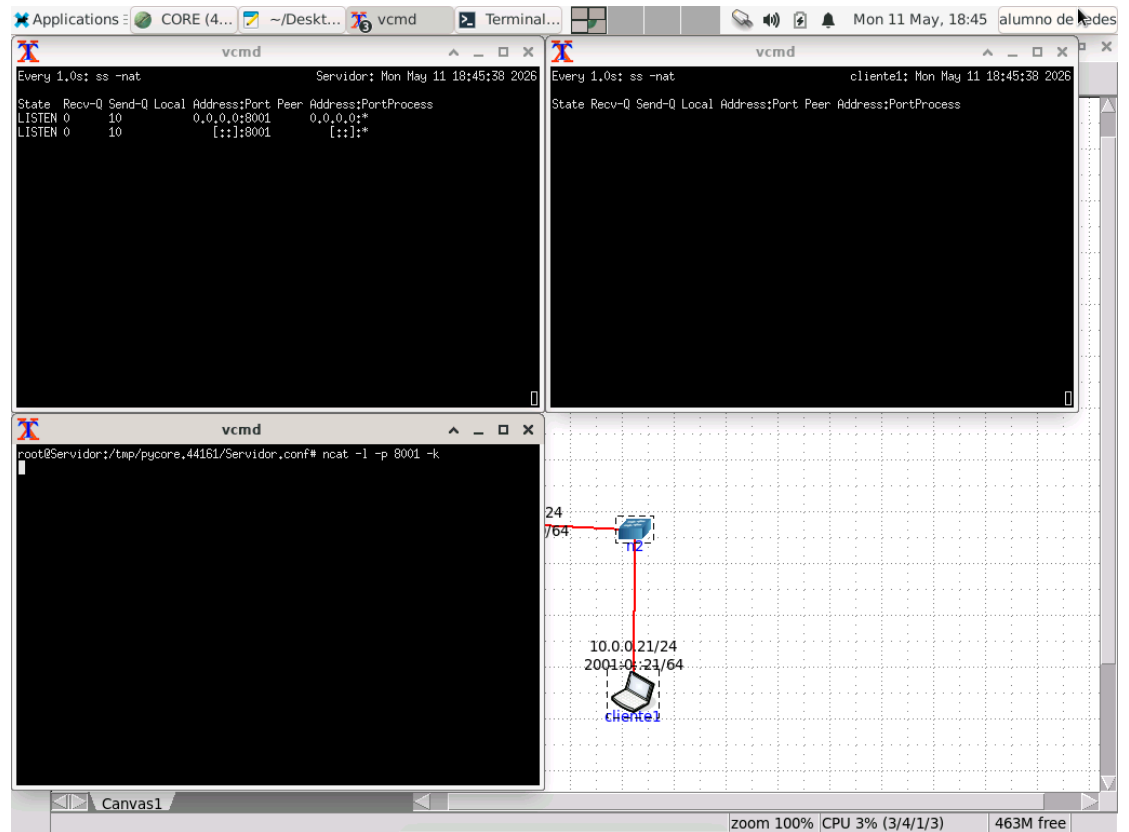
b.

- **Capa de Transporte: TCP.** Esto se observa directamente en la primera columna de la salida del comando ss en ambos equipos.
- **Capa de Aplicación: POP3** (Post Office Protocol version 3). El cliente está intentando conectarse al puerto de destino 110, el cual es el puerto estándar (well-known port) utilizado históricamente por este protocolo para la recuperación de correos electrónicos.

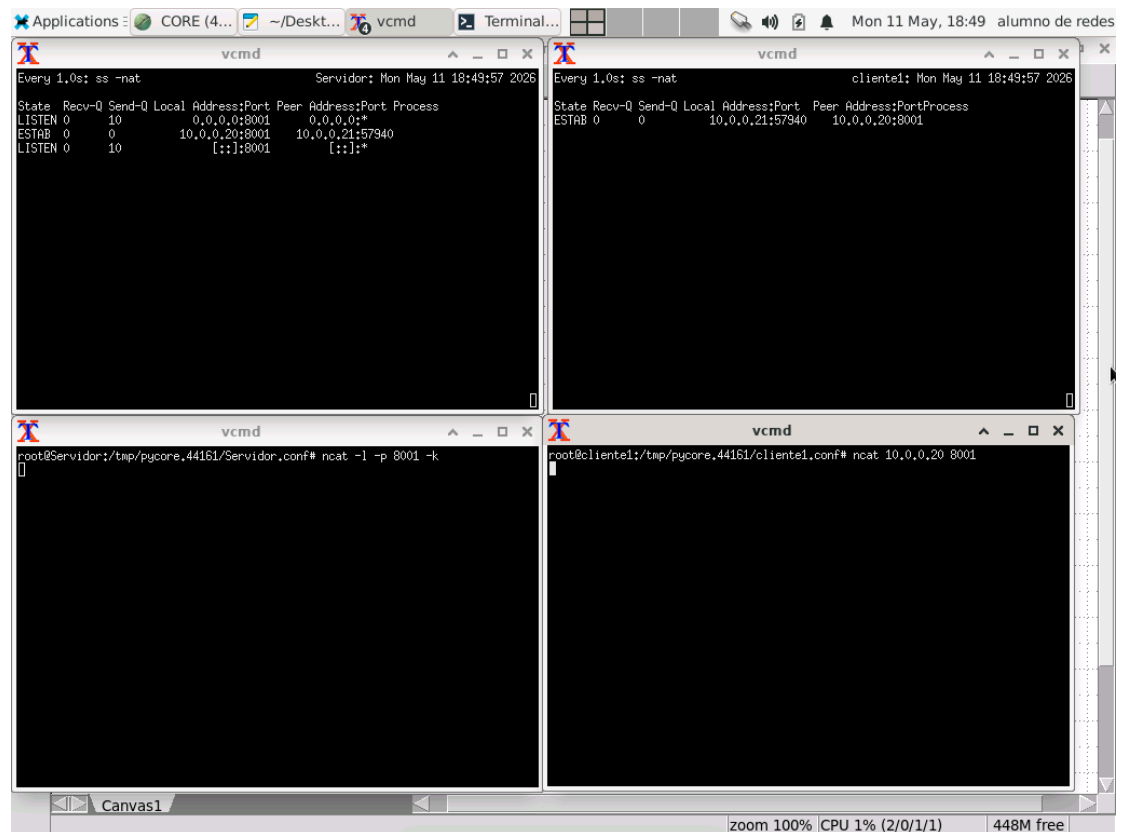
- c. El segmento que no está logrando llegar al cliente tendría los flags SYN y ACK seteados. Al estar en estado SYN-RECV, el servidor ya despachó este paquete como el segundo paso reglamentario del Three-way Handshake para acusar recibo (ACK) de la petición del cliente y, a su vez, enviar su propio número de secuencia inicial (SYN).

15.

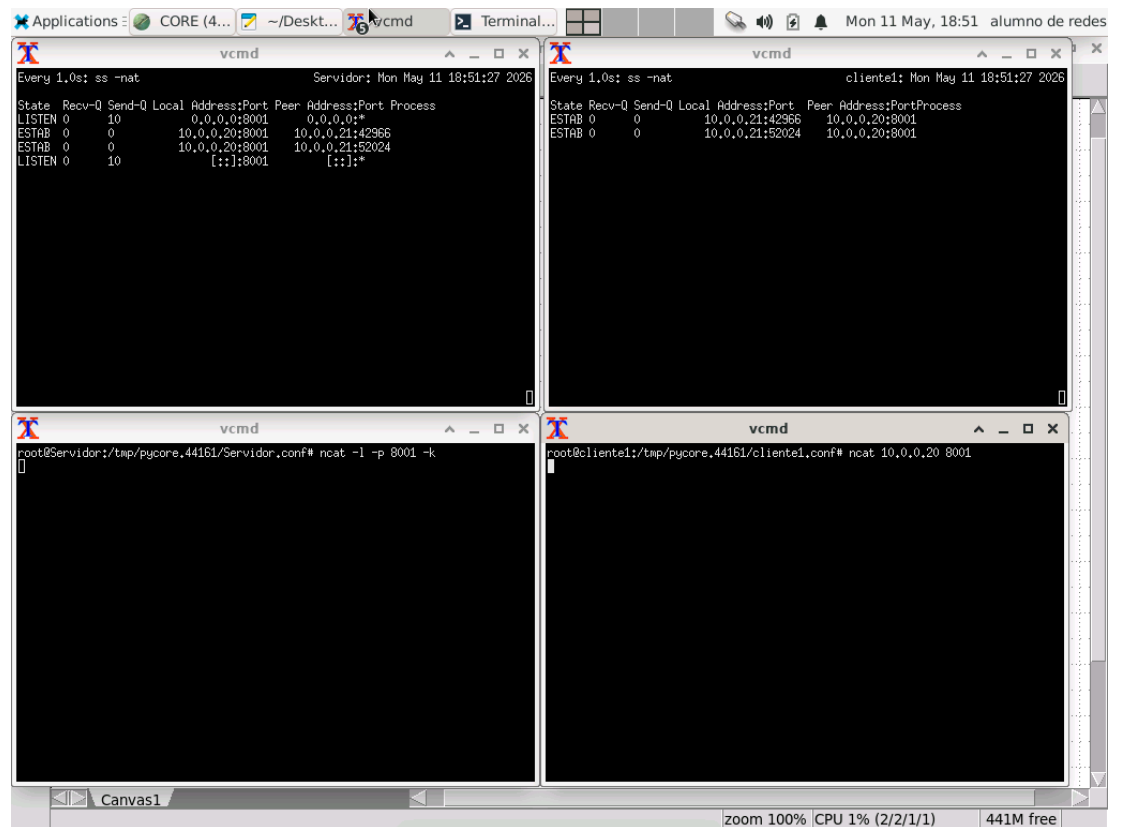




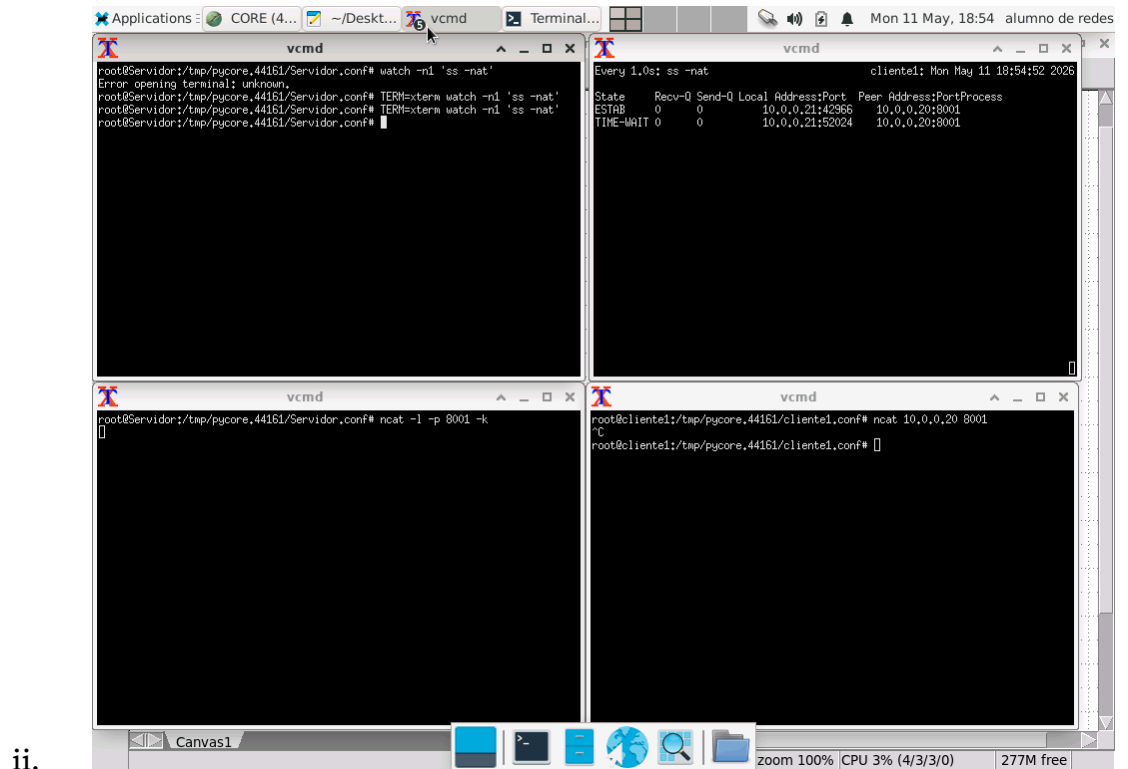
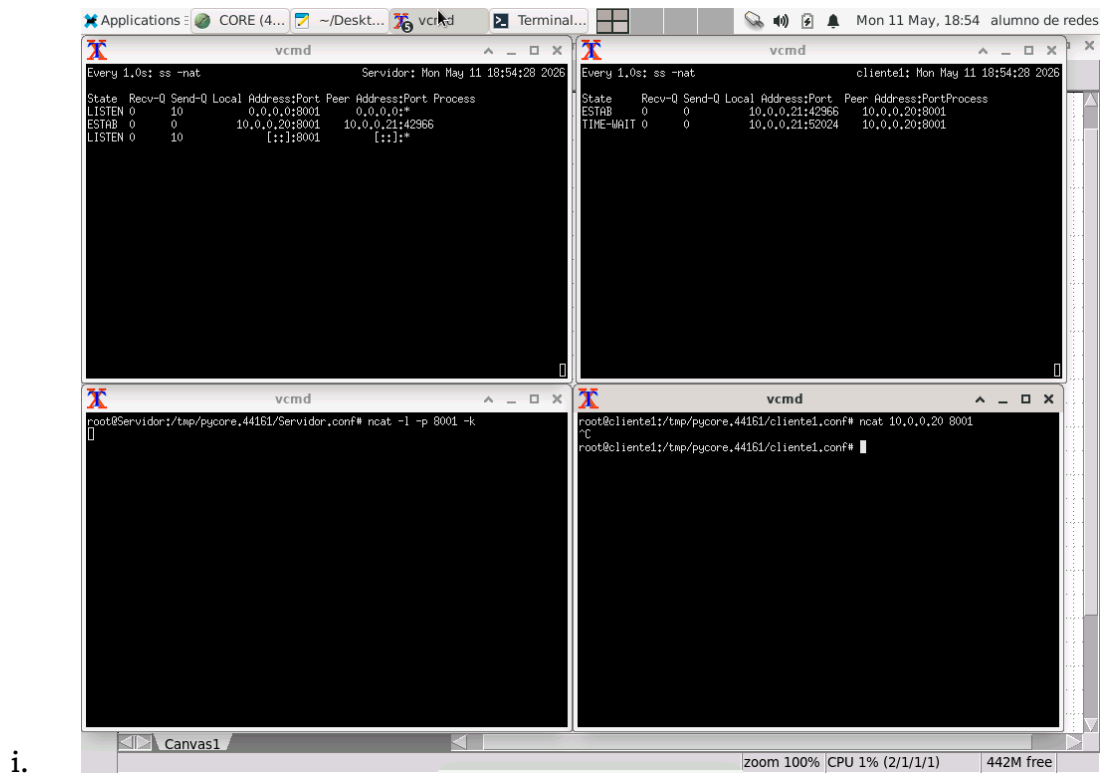
b.

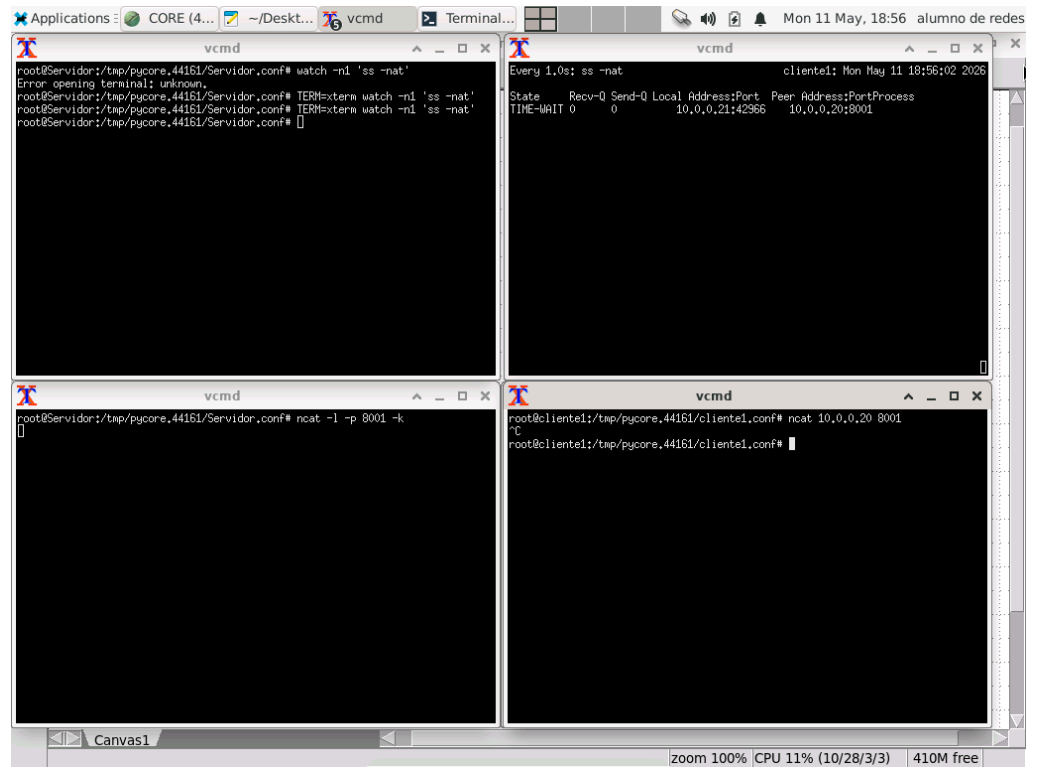


c.



- d.
- e. Sí, es totalmente posible porque TCP no identifica las conexiones solo por el puerto de destino, sino que utiliza una "4-tupla" o par de sockets. Una conexión se define por la combinación única de cuatro elementos: [IP Origen, Puerto Origen, IP Destino, Puerto Destino]. Al variar el Puerto Origen en cada nueva conexión que abre el cliente, la 4-tupla completa pasa a ser única. Así, la pila de red del sistema operativo sabe exactamente a qué proceso (a qué ventana de ncat) corresponde cada paquete de datos que llega, evitando que se mezclen.
- f.





iii.